

DFX4ML

Omitted for blind review

Keywords: FPGA, Partial Reconfiguration, Dynamic Function Exchange, PYNQ, Xilinx

Summary

DFX4ML-ARCH presents an FPGA architecture designed to enable partial reconfiguration for machine learning workloads. The project incorporates a self-reconfiguration mechanism, allowing the FPGA fabric to partially reprogram itself. It targets AMD FPGAs equipped with the ICAP3 interface and leverages the PYNQ framework for system integration.

Disclaimer: The technical content, architecture designs, implementation decisions, and all intellectual contributions in this report are entirely the work of the authors. Generative AI tools were used solely for grammar correction, readability enhancement, and summarization of the written text/code. No AI-generated technical content, code, or design decisions are included.

Template: This document uses the CERN ATS Note L^AT_EX template, available at <https://www.overleaf.com/latex/templates/cern-ats-note/yzjmgkrvtsqg>.

Contents

1	Project Overview	3
1.1	Project Structure	3
1.2	Naming Convention	4
2	Hardware Architecture	5
2.1	DFX Manager	6
2.1.1	DFX Manager's Reconfigurable Parameter	7
2.1.2	DFX Manager Core	8
2.2	DFX Streamer(s)	17
2.3	Buffer Sizing	18
2.4	DFX_CTRL	20
2.5	DFX Decoupler Controller	21
2.6	Other Relevant IPs	21

3	Driver Architecture	22
4	Build Procedure	24
4.1	Quick Start	24
4.2	ML Framework Integration	27
4.2.1	User Build Tcl	27
4.3	Board-Specific Script	29
5	HLS4ML Integration	29
5.1	The hls4ml_con Backend Plugin	30
5.2	The Hls4ml_build Orchestrator	31
5.3	Quick Start	34

1 Project Overview

DFX4ML is an FPGA architecture aiming to execute machine learning workloads on the FPGA by exploiting partial reconfiguration to fit the FPGA's resources to large machine learning models. The model will be divided into multiple pieces and then sequentially set up on the reconfigurable region to execute the machine learning workload.

In current version, the hardware and driver are built and tested under following specification:

1. FPGA model is tested on Zynq UltraScale+ (kria260)
2. Hardware model is composed in Vivado 2023.2
3. The target FPGA is running Ubuntu 22.04 with the PYNQ framework

This project is composed of sections:

1. **Hardware Architecture:** it describes the architecture idea, hardware implementation details, and reconfigurable parameters.
2. **Software Driver:** This driver mainly controls our hardware IP and Xilinx IP in the programmable logic. In this version, we provide only the PYNQ driver.
3. **Build Procedure:** This section provides our formal approach to enable DFX4ML-ARCH to integrate with existing Hardware Machine Learning Frameworks and to specific FPGA board.

1.1 Project Structure

Project Structure

```
.
+-- hw                                # for hardware resource
|   +-- bd_src                        # Vivado block design script
|   |   +-- dfx4ml                    # static and pr connection script
|   |   +-- dfx_region                # pr block design script
|   |   +-- dfx_unified               # static region block design script
|   +-- build_script                 # block design for specific board script
|   |   +-- kv260                     # KV260 board build + constraints
|   |   +-- custom                    # custom/template board build
|   +-- ip_src                       # verilog ip script
|       +-- dfx_axi_lite_shut
|       +-- dfx_decup_ctrl
|       +-- dfx_icap
|       +-- dfx_mng
|       +-- dfx_stream_aligner_m2s
|       +-- dfx_stream_aligner_s2m
|       +-- dfx_streamer
|       +-- dfx_streamer_mreset
|       +-- dfx_streamer_mshut
```

```

|      +-- dfx_streamer_s2m
|      +-- dfx_streamer_s2m_free
|      +-- dfx_streamer_sreset
|      +-- dfx_streamer_sshut
|      +-- fourSp
+-- lib                                # python library for hw and sw generation
|   +-- hls4ml_build                  # ML -> hls4ml -> dfx4ml orchestrator
|   +-- hls4ml_con                    # hls4ml -> dfx4ml backend plugin
|       +-- passes                    # custom hls4ml optimizer passes
|       +-- templates                # dfx-modified kernel templates
+-- sw                                # for software resource
    +-- driver                        # PYNQ DefaultIP drivers

```

1.2 Naming Convention

DFX4ML-ARCH adopts the following naming conventions across its Python and Verilog codebase.

- **Python classes and Verilog module names** use *Pascal_Snake_Case* (e.g., `Hw_Build_Helper`, `Dfx_Streamer`).
- **Python variables and methods** use *snake_case* (e.g., `build_folder_path`, `run_build()`).

Note that this convention is not strictly enforced throughout the entire codebase. Some legacy modules were written before this rule was established and may not fully conform. A refactoring pass to align these modules is planned for a future minor version.

2 Hardware Architecture

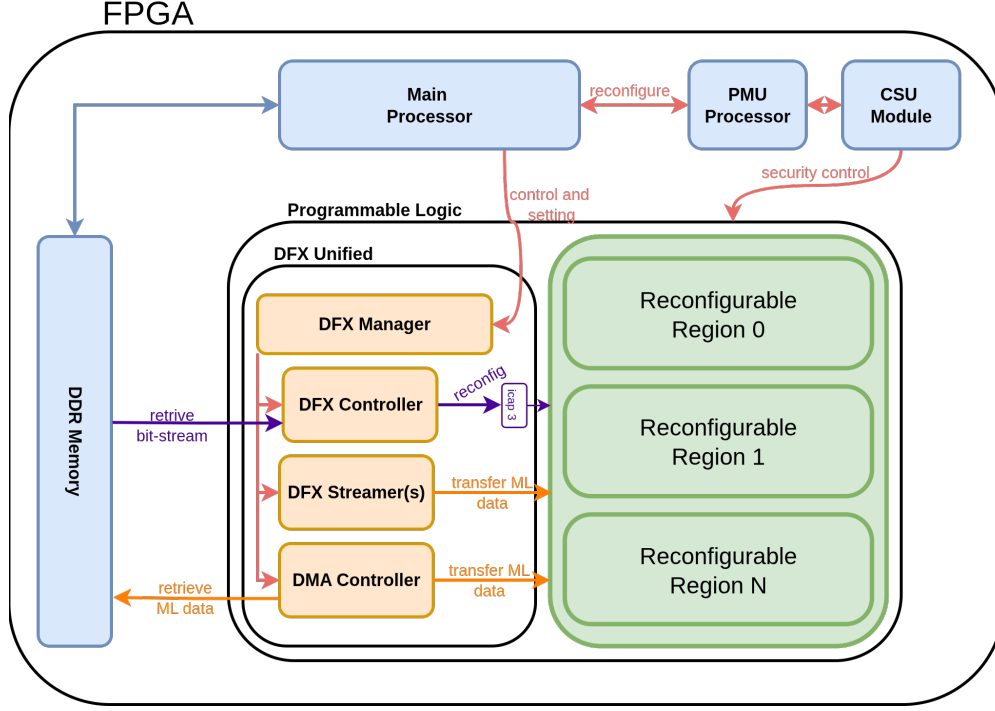


Figure 1: DFX4ML-ARCH overall architecture

Fig.1 illustrates the overall hardware architecture of DFX4ML-ARCH. The hardware architecture is composed of 4 major parts:

1. **Main Processor.** This project focuses on the Xilinx SoC with the PYNQ framework. The PYNQ framework allows users to manage memory allocation and initialization. Moreover, the processor is responsible for controlling Programmable Logic and communicating with the PMU processor.
2. **PMU and CSU.** The Platform Management Unit (PMU) and Control Security Unit (CSU) are the Xilinx SoC's management systems. The PMU is a dedicated processor used to manage the system's routine tasks and secure the system at the hardware level. In this project, we have to command the PMU to orchestrate the Programmable Logic permissions.
3. **Programmable Logic.** To support partial reconfiguration, the DFX4ML-ARCH is divided into 2 region:
 - (a) **Static Region.** The region that is always active and contains the control logic and temporary memory storage. We propose four main components:
 - i. **DFX Manager:** it manages all PL components: (1) commands the DFX Controller, (2) controls the DFX Streamer, (3) controls the DMA Controller,

- (4) profiles the system, (5) interrupts the processor, and (6) receives commands and metadata from the processor.
 - ii. **DFX Controller:** This Xilinx IP receives commands from the DFX Manager. When it is triggered, the system loads the partial bitstream file from main memory (DDR RAM) and reprograms the partial region through the ICAP3 port.
 - iii. **DFX Streamer(s):** it is the storage based on BRAM (on-chip memory) used to hold data temporarily while the partial region is reconfiguring.
 - iv. **DMA Controller:** this Xilinx IP is used to transfer data from DDR RAM directly to the partial region.
- (b) **Reconfigurable Region (RP).** This region is dedicated to machine learning workload. The input/output only comes from the AXI-Stream.

2.1 DFX Manager

DFX Manager is the IP used to orchestrate the entire Programmable Logic function. Fig. 2 illustrates the overall components in the manager.

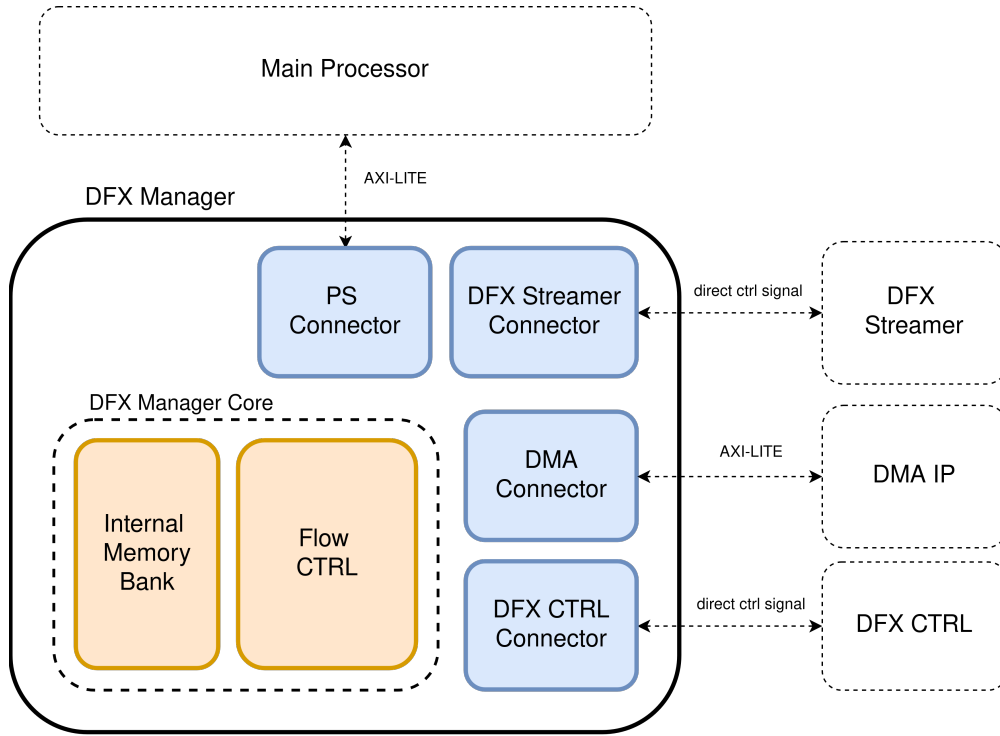


Figure 2: DFX Manager

The DFX Manager is composed of 5 major components:

1. **PS Connector:** It communicates with the main processor (PS) through the AXI-LITE slave protocol. The code is at `hw/ip_src/dfx_mng/s_axi_{read,write}.v`

2. DFX Streamer Connector: The connector directly passes signals from the DFX Manager Core through control signals to trigger load/store operations, and also receives the data-holding status from the streamers.
3. DMA Connector: The DFX Manager Core requires multiple steps to set up the DMA controller properly; therefore, the DMA connector is responsible for managing the sequence of AXI-LITE packets sent to the DMA IP. The code is at `hw/ip_src/dfx_mng/m_axi_{read,write}.v`
4. DFX CTRL Connector: The connector directly passes signals from the DFX Manager Core through control signals to trigger the DFX CTRL IP to decouple and reconfigure the partial region. This connector also receives data back from the target IP to indicate that the reconfiguration is finished.
5. DFX Manager Core: it contains the internal register memory used to control the flow of the system and provide data for the slave IPs. Moreover, Flow CTRL is responsible for orchestrating the slave IPs to work in the correct order.

2.1.1 DFX Manager’s Reconfigurable Parameter

The DFX Manager IP is parameterized at synthesis time via Verilog `parameter` declarations, allowing the designer to tailor its resource usage and functional scope to the target platform. Table 1 summarises all exposed parameters.

Table 1: DFX Manager Reconfigurable Parameters

Parameter	Default	Description
GLOBAL		
<code>GLOB_ADDR_WIDTH</code>	32	AXI address bus width.
<code>GLOB_DATA_WIDTH</code>	32	AXI data bus width.
<code>NUM_REGION</code>	2	Number of independently managed DFX reconfigurable regions.
BANK 0 — CONTROL & STATUS		
<code>BANK0_CONTROL_WIDTH</code>	4	The width of the control register
<code>BANK0_STATE_BIT_LEN</code>	4	state register width (recon_state, exec_state).
<code>BANK0_QUERY_BIT_LEN</code>	32	Query register bit width (bank0)
BANK 1 — SLOT & DMA DESCRIPTOR		
<code>BANK1_INDEX_WIDTH</code>	3	Session-index width; supports 2^n slots (default: 8).
<code>BANK1_DATA_ADDR_WIDTH</code>	32	DMA start-address width per session.
<code>BANK1_DATA_SIZE_WIDTH</code>	26	DMA transfer-size width (max 2^{26} bytes $\approx$ 64 MB).
<code>BANK1_RM_SELECT_WIDTH</code>	2	RM-select width; encodes $N_{VS} \times M_{RM/VS}$ module variants.

Table 1: DFX Manager Reconfigurable Parameters (continued)

Parameter	Default	Description
<code>BANK1_PROFILE_RECON_WID</code>	32	Reconfiguration phase profiling counter width.
<code>BANK1_PROFILE_EXEC_WIDT</code>	32	Execution phase profiling counter width.
<code>BANK1_DATA_POOL_MASK_WI</code>	8	DFX Streamers (MGSs) selecting bit width

2.1.2 DFX Manager Core

Register Architecture (internal register bank)

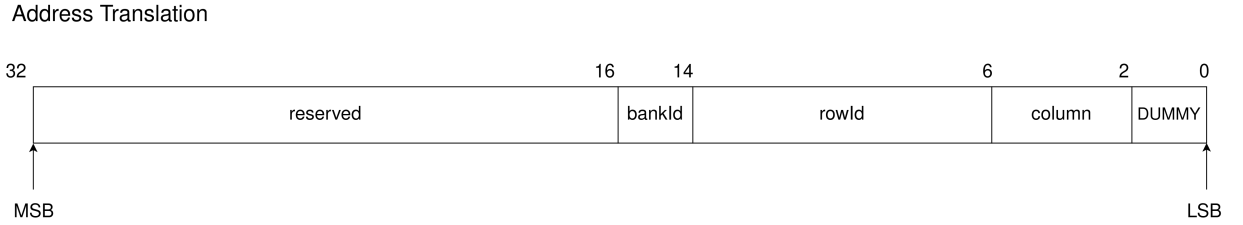


Figure 3: DFX Manager Core’s internal register address translation

We propose the register architecture that resides in the internal memory bank. Users can access these via memory-mapped I/O. The TCL build script sets the default address at `0xA000_0000` with a 64 KB address range.

With a 64 KB range, the address space is categorized into 3 main sections:

1. **bankId** (bits 14–15): the system supports up to 4 banks. Each bank is a group of rows that belong to the same functional group.
2. **rowId** (bits 13–6): each bank supports up to 256 rows. A row is a group of registers; however, different rows in the same bank do not have to follow the same schema.
3. **column** (bits 5–2): each row supports up to 16 registers. Each register in the row does not have to be the same size. However, the register size must not exceed 4 bytes (32 bits).

Fig. 4 illustrates the DFX Manager’s internal memory. The system is composed of two memory banks: Bank 0 is dedicated to IP control, the debugging system, overall running configuration, interrupt settings, and flow control memory; Bank 1 is composed of several rows sharing the same register pattern, used to store data and metadata for reconfigurable module initialization and slave static IP commands.

Bank 0 — Global Control Registers

This bank is mainly used to control the flow control system and slave IP identifiers. It provides 13 registers accessible from the processor (PS) via the AXI-LITE protocol. Note that each Bank 0 register occupies a full row (64 B stride), since bits[13:6] select the row with no column multiplexing inside Bank 0. The code is at [hw/ip_src/dfx_mng/dfx_mng_core.v](#)

Base offset: 0x0000

Table 2: Bank 0 — Global Control Registers

Address	Register	Width	Access	Description
0x0000	CTRL_REG	BANKO_CONTROL_WIDTH	WO	Control register (write-only). 0x0 = CLEAR (reset all state machines and counters) 0x1 = SHUTDOWN (halt gracefully) 0x2 = START (begin processing)
0x0040	MAIN_STATE	BANKO_STATE_BIT_LEN	RO	Main flow state machine output: 0x0 = SHUTDOWN 0x1 = PROCESS 0x2 = PRE_SHUTDOWN
0x0080	RECON_STATE	BANKO_STATE_BIT_LEN	RO	Reconfiguration sub-state machine (runs in parallel with EXEC_STATE): 0x0 = SHUTDOWN 0x1 = REPROG 0x2 = W4SLAVERESET 0x3 = W4SLAVEOP 0x4 = FIN_SYNC

Table 2: Bank 0 — Global Control Registers (continued)

Address	Register	Width	Access	Description
0x00C0	EXEC_STATE	BANK0_STATE_BIT_LEN	RO	Execution sub-state machine (runs in parallel with RECON_STATE): 0x0 = SHUTDOWN 0x1 = INITIALIZE_PR_CTRL 0x2 = CLEAR_MGS 0x3 = INITIALIZE_MGS 0x4 = INITIALIZE_DMA 0x5 = SET_DMA_LOAD 0x6 = SET_DMA_STORE 0x7 = TRIGGERING 0x8 = WAIT4FIN 0x9 = FIN_SYNC
0x0100	LAST_SESSION	BANK1_INDEX_WIDTH	R/W	Index of the last session in the execution round. The main machine stops cycling when the current session index equals this value and the query budget is exhausted.
0x0140	CUR_QUERY	BANK0_QUERY_BIT_LEN	RO	Current query counter. Incremented by AMT_QUERY_PER_ITER after each full session round. Cleared on CLEAR or START.
0x0180	AMT_QUERY	BANK0_QUERY_BIT_LEN	R/W	Total query budget. The system shuts down automatically when CUR_QUERY reaches this value.
0x01C0	AMT_QUERY_PER_ITER	BANK0_QUERY_BIT_LEN	R/W	Number of queries processed per iteration (per full session round). Added to CUR_QUERY on each round completion.
0x0200	DMA_IP_ADDR	GLOB_ADDR_WIDTH	R/W	Base address of the DMA IP in the memory-mapped peripheral space. Used by the DMA Connector to issue AXI-LITE transactions.

Table 2: Bank 0 — Global Control Registers (continued)

Address	Register	Width	Access	Description
0x0240	PR_IP_ADDR	GLOB_ADDR_WIDTH	R/W	Base address of the PR-controlled IP (e.g., the ML accelerator) in the peripheral space. Used by the PR CTRL Connector to initialize the IP via <code>batch_size</code> and <code>ap_start</code> .
0x0280	INTR_ENA	1 bit	R/W	Interrupt enable. Set to 1 to assert the interrupt line when all queries finish.
0x02C0	INTR_STATUS	1 bit	RO	Interrupt status. Set to 1 when <code>CUR_QUERY = AMT_QUERY</code> . Cleared on CLEAR or START.
0x0300	MPERF	BANK0_QUERY_BIT_LEN	R/W	Global performance counter. Increments every clock cycle while <code>RECON_STATE</code> or <code>EXEC_STATE</code> is not in SHUTDOWN, measuring total engine-busy cycles across a run. Writable from the PS while the system is in SHUTDOWN (like the other R/W Bank 0 registers); cleared on hardware reset.

Bank 1 — Reconfigurable Module Slot Registers

This bank stores per-session data and metadata. Each row (session slot) follows a uniform 12-register pattern and contains DMA transfer parameters, profiling counters, RM selection fields, I/O stream masks, and a linked-list `NEXT_SESSION` pointer. The session execution order is defined by the `NEXT_SESSION` chain rather than contiguous row indices. The code is at `hw/ip_src/dfx_mng/dfx_mng_core.v`

*Base offset: $0x4000 + 0x40 * \text{session}_i$*

Slot Offset	Register	Width	Access	Description
DMA TRANSFER — SOURCE				
+0x00	DMA_SRC_ADDR	BANK1_DATA_ADDR_WIDTH	R/W	Source memory address. Address in system memory from which the DMA engine reads data for this session.
+0x04	DMA_SRC_SIZE	BANK1_DATA_SIZE_WIDTH	R/W	Source transfer size in bytes for this session.
DMA TRANSFER — DESTINATION				
+0x08	DMA_DES_ADDR	BANK1_DATA_ADDR_WIDTH	R/W	Destination memory address. Address in system memory where the DMA engine writes output data for this session.
+0x0C	DMA_DES_SIZE	BANK1_DATA_SIZE_WIDTH	R/W	Destination transfer size in bytes for this session.
PROFILING COUNTERS				
+0x10	PROF_RECON	BANK1_PROFILE_RECON_WIDTH	R/W	Reconfiguration profiler counter. Auto-incremented every cycle while RECON_STATE is in REPROG, W4SLAVERESET, or W4SLAVEOP. PS may write to reset or preset the counter.
+0x14	PROF_EXEC	BANK1_PROFILE_EXEC_WIDTH	R/W	Execution profiler counter. Auto-incremented every cycle while EXEC_STATE is in any active (non-SHUTDOWN / non-FIN_SYNC) state. PS may write to reset or preset the counter.
RM SELECTION				
+0x18	VS_RM_RECON_SELECT	BANK1_RM_SELECT_WIDTH	R/W	Selects which reconfigurable module to program during RECON. Forwarded to the DFX CTRL IP as the dfx_rm_program signal.
+0x1C	VS_RM_EXEC_SELECT	BANK1_RM_SELECT_WIDTH	R/W	Selects which RM the execution pipeline targets. Allows a different RM to be reconfigured while another executes.
STREAM MASKS				

Slot Offset	Register	Width	Access	Description
+0x20	LOAD_MASK	BANK1_DATA_POOL_MASK_WIDTH	R/W	Bit mask of input streamers that must be loaded before this session may execute. Bit 0 (LSB) is reserved for DMA; bits 1–7 correspond to DFX Streamer channels 1–7.
+0x24	STORE_MASK	BANK1_DATA_POOL_MASK_WIDTH	R/W	Bit mask of output streamers that must store data after execution. Bit 0 (LSB) is reserved for DMA; bits 1–7 correspond to DFX Streamer channels 1–7.
+0x28	COMPLETE_MASK	BANK1_DATA_POOL_MASK_WIDTH	R/W	Tracks which store operations have completed (same bit layout as STORE_MASK). Bits are ORed in automatically as streamers finish (dfx_stream_fin). The session is complete when COMPLETE_MASK = STORE_MASK.
SESSION LINKED LIST				
+0x2C	NEXT_SESSION	BANK1_INDEX_WIDTH	R/W	Index of the next session slot to execute after this one. Forms a singly-linked list of sessions; a round ends when the current slot index equals LAST_SESSION in Bank 0.

Access Permission Legend

R/W	Read / Write	Register can be both read and written by the host.
RO	Read Only	Register is read-only; writes are ignored.
WO	Write Only	Register is write-only; reads return undefined values.

Flow Control (state machine)

Flow Control orchestrates the slave IP steps — DFX Controller, DMA IP, and DFX Streamer (formerly Magic Streamer) — to execute initialization and operation commands sequentially

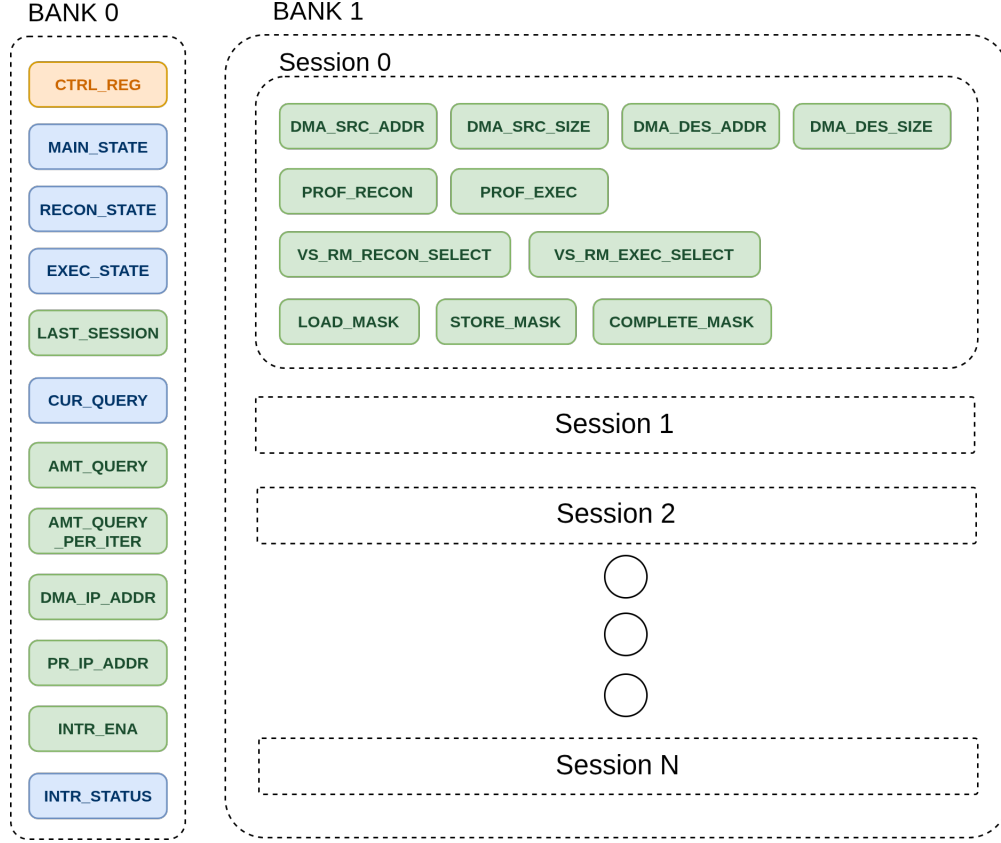


Figure 4: DFX Manager Core's internal register overview

at the cycle-accurate level. The code is at [hw/ip_src/dfx_mng/dfx_mng_core.v](#).

The DFX Manager Core runs three concurrent, loosely-coupled state machines whose outputs are readable from Bank 0 via [MAIN_STATE](#), [RECON_STATE](#), and [EXEC_STATE](#). Fig. 5 illustrates the high-level state diagram.

Table 4: DFX Manager Flow Control — State Machine Summary

State	Description
1. MAIN STATE MACHINE (MAIN_STATE)	
SHUTDOWN	Idle; Bank 0/1 registers are PS-writable. Waits for CTRL_REG = CTRL_START .
PROCESS	Launches RECON and EXEC in parallel; holds until both reach FIN_SYNC simultaneously.
PRE_SHUTDOWN	Single-cycle housekeeping — advances the session pointer (NEXT_SESSION), increments DMA addresses, then either loops back to PROCESS or returns to SHUTDOWN . Asserts INTR_STATUS if INTR_ENA is set and all queries are complete.

Table 4: DFX Manager Flow Control — State Machine Summary (continued)

State	Description
2. RECONFIGURATION SUB-MACHINE (RECON_STATE)	
REPROG	Reads <code>VS_RM_RECON_SELECT</code> from Bank 1. • If 0: skip to <code>FIN_SYNC</code> (bypasses <code>W4SLAVERESET</code> and <code>W4SLAVEOP</code>). • Otherwise: drives <code>dfx_rm_program</code> → <code>W4SLAVERESET</code>.
W4SLAVERESET	Waits for <code>dfx_rm_nreset</code> to deassert.
W4SLAVEOP	Waits for <code>dfx_rm_nreset</code> to reassert (reconfiguration complete).
FIN_SYNC	Holds until EXEC also reaches <code>FIN_SYNC</code> ; both return to <code>SHUTDOWN</code> together.
3. EXECUTION SUB-MACHINE (EXEC_STATE)	
PR_CTRL_REQ _SKIP_CHECK	Entry state; reads <code>VS_RM_EXEC_SELECT</code> from Bank 1. • If 0: skip to <code>CLEAR_MGS</code>. • Otherwise: arm PR CTRL sequencer → <code>INITIALIZE_PR_CTRL</code>.
INITIALIZE _PR_CTRL	Sends <code>batch_size</code> and <code>ap_start</code> to the PR-controlled IP (<code>PR_IP_ADDR</code>); advances on <code>AP_START</code> completion.
CLEAR_MGS	One-cycle reset of DFX Streamer channels in <code>LOAD_MASK</code> .
INITIALIZE_MGS	One-cycle init of DFX Streamer channels in <code>STORE_MASK</code> .
INITIALIZE_DMA	Resets the DMA interrupt flag.
SET_DMA_LOAD	Programs DMA source address and size (<code>DMA_SRC_ADDR</code> , <code>DMA_SRC_SIZE</code>); skipped if <code>LOAD_MASK[0] = 0</code> .
SET_DMA_STORE	Programs DMA destination address and size (<code>DMA_DES_ADDR</code> , <code>DMA_DES_SIZE</code>); skipped if <code>STORE_MASK[0] = 0</code> .
TRIGGERING	One-cycle state delayed (reserved for future used).
WAIT4FIN	Waits until <code>COMPLETE_MASK = STORE_MASK</code> .
FIN_SYNC	Holds until RECON also reaches <code>FIN_SYNC</code> ; both return to <code>SHUTDOWN</code> together.

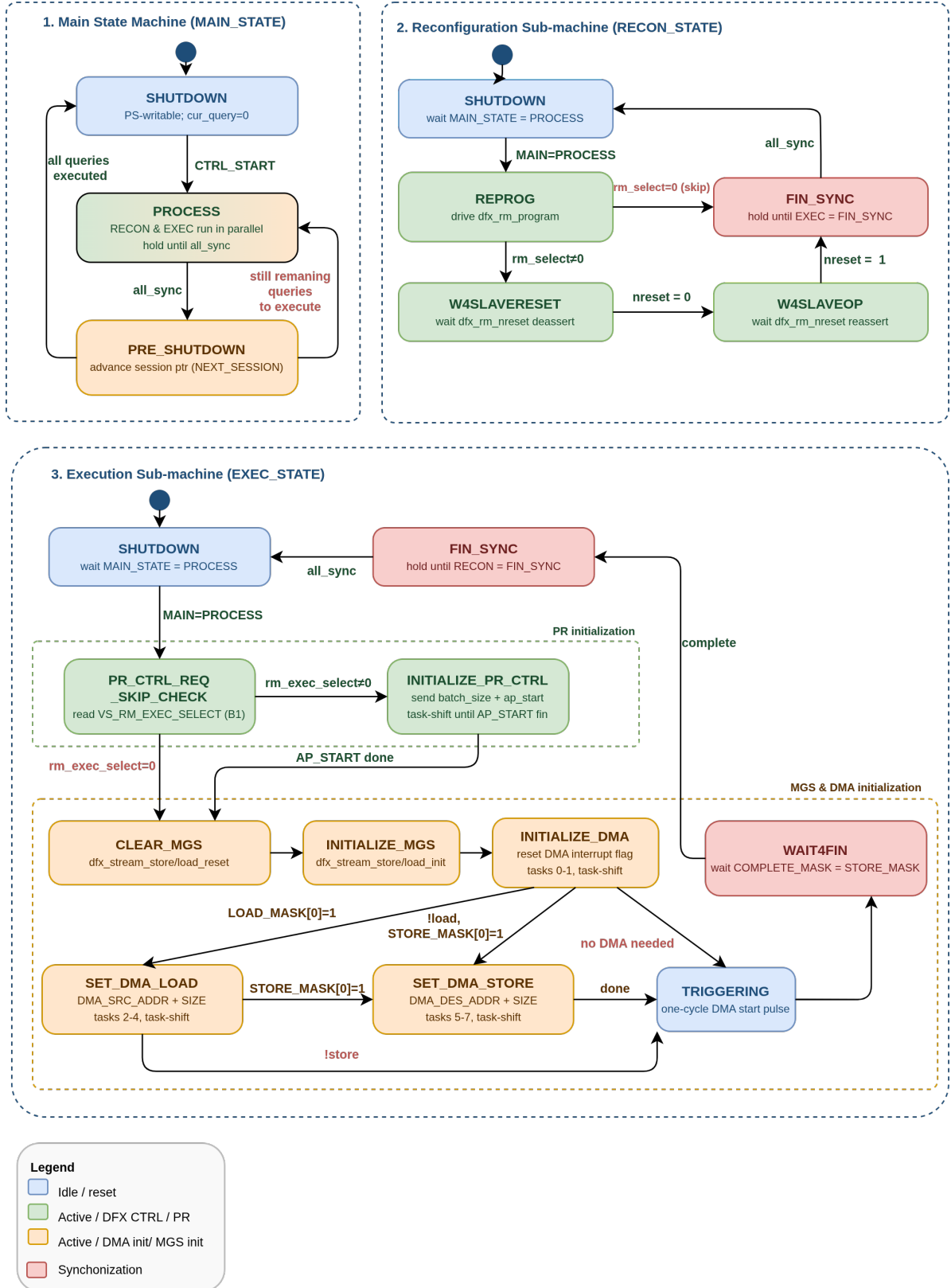


Figure 5: DFX Manager flow control (state diagram)

2.2 DFX Streamer(s)

DFX Streamer(s) is the temporary storage residing in the Programmable Logic static region and was designed to utilize BRAM resources for storage and a small number of flip-flops to hold its transfer metadata. The code is at [hw/ip_src/dfx_streamer/dfx_streamer.v](#)

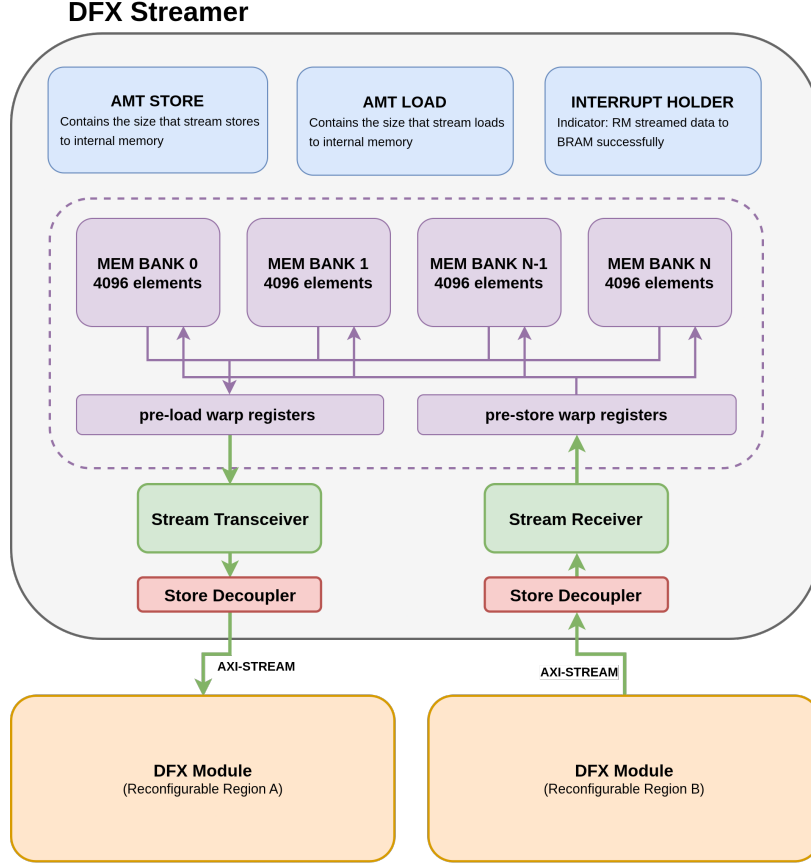


Figure 6: DFX Streamer Architecture

Fig. 6 illustrates the overall architecture of the DFX Streamer. The **meta-data blocks** (blue boxes) reside in the static region, using flip-flops to track the amount of load/store operations and RM status (INTERRUPT HOLDER). Meanwhile, the **MAIN MEMORY** (purple block) also resides in the static region, but uses BRAM to store the temporary data.

The DFX Streamer's main memory packs 4096 entries per bank to maximise resource utilisation for each URAM. If entries are not packed, Vivado cannot decide to merge two memory banks into a single URAM/BRAM, even when they are not accessed simultaneously.

To address the latency problem, we pipeline the URAM/BRAM data transfer (both load and store). On the store side, we add an additional register equal to the wrap size to buffer data coming from the dynamic region. On the load side, we add an additional register equal to wrap size $\times$ number of memory banks; the pipeline supports back-pressure in case the ML model is not ready to receive data from the memory bank.

In contrast, the Stream Transceiver and Stream Receiver connection ports are open to the partial region via the **AXI-STREAM** interface. In the current version, the AXI-STREAM

tkeep and **tlast** signals are **mandatory**. If they are not set up, the system will get stuck.

To support the parallel execution between (**Latency Hiding Technique**), the stream transceiver and stream receiver have their own dedicated DFX Decoupler (red box); therefore, it enables the DFX Streamer operating while there are reconfiguring modules.

2.3 Buffer Sizing

The buffer sizing of the DFX Streamer(s) is computed automatically from the partitioned model's stream geometry. The overall procedure is summarised in Algorithm 1. It works in four stages:

1. compute the bank geometry of every stream;
2. hand each stream a physical streamer for the phases it is alive, reusing a free streamer of the same size and region whenever possible;
3. spend the remaining bank budget on the streamer with the smallest capacity;
4. report the result.

Algorithm 1 DFX streamer buffer allocation

Require: streams (each with $shape$, $precision$, $region$, $allocPhase$, $freePhase$); bank budget N ; phase count P ; bank width BW and depth BD

Ensure: for every streamer: its bank count and buffered-query capacity

```
1: Stage 1: per-stream geometry
2: for each stream  $s$  do
3:    $entries \leftarrow$  product of all but the last dimension of  $s.shape$   $\triangleright$  items per query
4:    $bits \leftarrow s.precision \times$  (last dimension of  $s.shape$ )
5:    $s.banks \leftarrow \lceil bits/BW \rceil$   $\triangleright$  banks needed for one item
6:    $s.cap \leftarrow \lfloor BD/entries \rfloor$   $\triangleright$  queries one bank set can hold
7:   if  $s.cap = 0$  then
8:     error: “one query does not fit in a bank set”
9:   end if
10: end for

11: Stage 2: allocate streamers across phases (reuse when possible)
12:  $free \leftarrow \emptyset$ ;  $using \leftarrow \emptyset$ ;  $streamers \leftarrow \emptyset$ 
13: for  $phase \leftarrow 0$  to  $P - 1$  do
14:   for all stream  $s$  with  $s.allocPhase = phase$  do
15:     look for a free streamer  $f$  with  $f.banks = s.banks$  and  $f.region = s.region$ 
16:     if such  $f$  exists then
17:       attach  $s$  to  $f$ ;  $f.freePhase \leftarrow s.freePhase$ ; move  $f$  from  $free$  to  $using$ 
18:     else
19:       create new streamer ( $banks = s.banks$ ,  $region = s.region$ ,  $mul = 1$ , holding
20:        $s$ )
21:       add it to  $using$  and to  $streamers$ 
22:     end if
23:   end for
24:   move each  $using$  streamer whose  $freePhase = phase$  back to  $free$   $\triangleright$  data no longer
25:   needed
26: end for
27: if  $streamers = \emptyset$  then
28:   return  $\{streamers, banksUsed = 0\}$ 
29: end if

28: Stage 3: grow buffers within the bank budget (water-filling)
29: while true do
30:    $banksUsed \leftarrow \sum_{d \in streamers} d.banks \times d.mul$ 
31:    $b \leftarrow$  streamer with the smallest capacity  $d.mul \times d.cap$   $\triangleright$  the bottleneck
32:   if  $banksUsed + b.banks > N$  then
33:     break  $\triangleright$  next growth would exceed the budget
34:   end if
35:    $b.mul \leftarrow b.mul + 1$   $\triangleright$  give the bottleneck one more bank set
36: end while

37: Stage 4: report
38: return  $\{streamers, banksUsed, \min_{d \in streamers} (d.mul \times d.cap)\}$ 
```

2.4 DFX_CTRL

DFX_CTRL is the Xilinx DFX IP used to retrieve the bitstream file from DDR memory and reconfigure the partial reconfigurable region using the ICAP3 port. To achieve this, we need to configure the IP's internal memory to match our requirements.

Figure 5: Register Banks within a Virtual Socket Manager

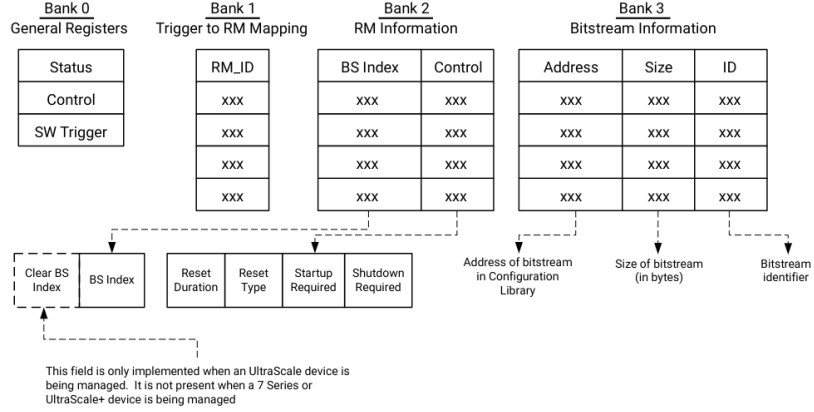


Figure 7: DFX Controller Architecture (copied from [1])

Fig. 6 [1] illustrates the internal memory of the DFX_CTRL IP. It is composed of 4 memory banks:

1. Bank 0 contains general registers enabling software to control the IP and check its status.
2. Bank 1 is the RM Mapping; each slot in this bank is a pointer to a slot in Bank 2.
3. Bank 2 contains RM information. The control field is the setting of the start-up/shutdown procedure, and our driver sets these criteria as follows:
 - (a) Reset Duration: 1 cycle
 - (b) Reset Type: active low
 - (c) Startup Required: No
 - (d) Shutdown Required: No

The BS Index field contains the index of the target slot in Bank 3.

4. Bank 3 contains the bitstream information:
 - (a) Address: the address of the bitstream file in DDR memory
 - (b) Size: the size of the bitstream file in bytes
 - (c) ID: bitstream identifier

In the case of multiple reconfigurable regions, the system will create Banks 0–3 equal to the number of the user's reconfigurable regions.

2.5 DFX Decoupler Controller

For debugging purposes, in the old version the DFX Controller IP owned the DFX decoupler, making it impossible for the PS to bypass control in some situations. We therefore redesigned the ownership of the DFX decoupler into a per-region mux (`dfx_decup_ctrl`), located at address `0xA001_0000`. The register maps to the `decup_and_ctrl_ps` port: bit 0 is the global source-select, and bits `[N:1]` carry the per-region PS decoupler values (one bit per reconfigurable region, where $N = \text{NUM_REGION}$). The resulting decoupler signal can be routed to `decup_store`, `decup_load`, or both on the target `Dfx_Streamer`. The register bit-field definition is shown in Table 5.

Table 5: DFX Decoupler Control Register (`0xA001_0000`) bit-field definition.

Bit	Access	Value	Description
0	WO	1	Decoupler source is the DFX Controller (used during partial reconfiguration).
		0	Decoupler source is the PS (used during debug/manual operation).
r+1	WO	1	PS decoupler value for region r is decoupled (isolated). Valid only when Bit 0 = 0 (PS source).
		0	PS decoupler value for region r is coupled (normal operation). Valid only when Bit 0 = 0 (PS source).

2.6 Other Relevant IPs

In DFX4ML, there are 3 other IPs that support system creation:

1. `icapWrap` is the module that serves as an interface for the DFX Controller to self-reconfigure. `hw/ip_src/dfx_icap/dfx_streamer.v`
2. `Stream_Master_Dummy` is the module used as a dummy plug on the partial region AXI-Stream master side. It is built to prevent unpredictable signals when the target reconfigurable module does not require certain interface ports. `hw/ip_src/dfx_streamer_mshut/streamer_mshut.v`
3. `Stream_Slave_Dummy` is similar to `Stream_Master_Dummy` but for the AXI-Stream slave side. `hw/ip_src/dfx_streamer_sshut/streamer_sshut.v`

3 Driver Architecture

Memory Address

Our build script sets a default base address of 0xA000_0000 for the DFX Unified IP, which includes DFX Mng IP, DFX Ctrl IP, DMA IP, PR Reset, and PR DECUP.

Table 6: DFX Unified IP Address Map (Base: 0xA000_0000)

IP	Offset	Absolute Address	Address Range	
DFX Ctrl	0x0_0000	0xA000_0000	0xA000_0000 0xA000_FFFF	—
PR DECUP	0x1_0000	0xA001_0000	0xA001_0000 0xA001_FFFF	—
PR Reset	0x2_0000	0xA002_0000	0xA002_0000 0xA002_FFFF	—
DMA	0x3_0000	0xA003_0000	0xA003_0000 0xA003_FFFF	—
DFX Mng	0x4_0000	0xA004_0000	0xA004_0000 0xA004_FFFF	—
PR Ctrl*	0x5_0000 $+ r \times 0x1_0000$	0xA005_0000 $+ r \times 0x1_0000$	0xA005_0000 0xA005_FFFF $+ r \times 0x1_0000$	—

* One instance per reconfigurable region; $r = 0, 1, \dots, N-1$.

Driver Component

This section illustrates the PYNQ driver overview used to command the DFX4ML-ARCH hardware.

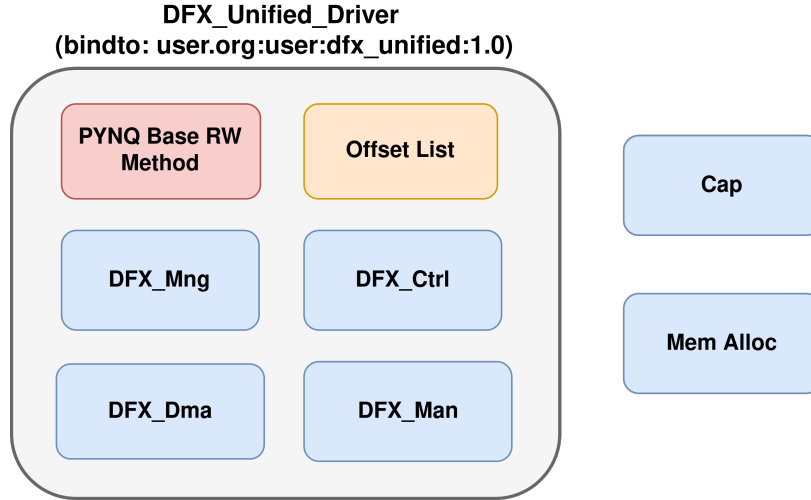


Figure 8: DFX4ML-ARCH Driver Overview

DFX_Unified_Driver: This class controls the DFX Unified IP’s operations. As described in the Hardware Architecture section, the DFX Unified IP is composed of 4 sub-IPs: DFX Manager, DFX Controller, DFX DMA, and DFX Streamer(s).

Therefore, **The Unified Driver** is composed of 4 sub-drivers (blue boxes): DFX Manager Driver to control the Reconfigurable Module’s flow; DFX Controller Driver to manage the partial bitstream; DMA Driver to manage the memory data transfer, used for debugging purposes only; and DFX Manual, used to manually decouple and reset the partial region, also for debugging purposes only.

the code is at

```

sw/driver/dfx_unified.py
sw/driver/dfx_mng.py
sw/driver/dfx_ctrl.py
sw/driver/dfx_dma.py
sw/driver/dfx_man.py

```

Cap is the helper function enabling designers to communicate with the PMU processor and reconfigure the `pcap_ctrl` register. After changing this register to 1, the FPGA will enable partial reconfiguration via the ICAP3 port. In contrast, if the register is set to 0, the FPGA will allow reconfiguration only from the processor (PS).

Mem Alloc is a helper function that facilitates users in allocating and copying data.

4 Build Procedure

4.1 Quick Start

We provide a Python interface that allows users to generate full and partial bitstream files (*.bin*), hardware handoff files (*.hwh*), DFX Ctrl configuration files, and PYNQ driver files.

Listing 1: Python Build Interface

```
1 from lib.hw_build import HwBuildHelper
2 from lib.sw_build import SwBuildHelper
3
4 # Streamer definitions (index 0 is always the DMA)
5 # load_width and store_width are in bytes
6 dfx_streamers = [
7     {"load_width": 4, "store_width": 4, "actual_width": 32, "
8         amount_row": 1024},
9     {"load_width": 4, "store_width": 4, "actual_width": 32, "
10         amount_row": 1024},
11 ]
12
13 # Region definitions: two regions forming a pipeline DMA->r0->r1->DMA
14 dfx_regions = [
15     {"load_streamers": [0], "store_streamers": [1]},
16     {"load_streamers": [1], "store_streamers": [0]},
17 ]
18
19 # RM schematics: rm_schematics[region_idx][rm_idx]
20 # load_io_map/store_io_map: list of (streamer_index,
21     kernel_port_index) pairs
22 rm_schematics = [
23     [{"load_io_map": [(0, 0)], "store_io_map": [(1, 0)]}],
24     [{"load_io_map": [(1, 0)], "store_io_map": [(0, 0)]}],
25 ]
26
27 # Hardware Build
28 hw_builder = HwBuildHelper(
29     build_folder_path = "./build_prj",
30     dfx_root_path = "../..",
31     board = "custom",
32     board_build_tcl = "./kv260/board_build.tcl",
33     constraint_xdc = "./kv260/constraint.xdc",
34     user_repo_path = "",
35     user_rm_build_tcl_path = "",
36     req_gen_ip = 1,
37     num_core = 4,
38     clk_frq = 99999001,
39     rm_index_width = 2,
40     dfx_streamers = dfx_streamers,
```



```

38     dfx_regions          = dfx_regions ,
39     rm_schemetics        = rm_schemetics ,
40     test_mode            = 1,
41     vivado_path           = "<vivado path>",
42     export_folder_path    = "./export"
43 )
44 hw_builder.run_build()
45 hw_builder.package_export_files()
46
47 # Software Build
48 # All parameters are derived from hw_builder; pass export_folder_path
49 # num_pr_region, rm_index_width, or num_streamer explicitly to
49 override.
50 sw_builder = SwBuildHelper(hw_builder = hw_builder)
51 sw_builder.package_export_file()

```

The following example (Listing 1) demonstrates how to start building the DFX4ML-ARCH system. The reference file is at [example/muti_region_test/multi_region_test.ipynb](#). Lines 1–2 import the Hardware and Software Build Helpers. Lines 6–22 define the three structured configuration objects (`dfx_streamers`, `dfx_regions`, `rm_schemetics`). Lines 25–45 instantiate `HwBuildHelper` and invoke synthesis and implementation; the final lines instantiate `SwBuildHelper` and export the PYNQ driver files. `SwBuildHelper` derives its parameters (`export_folder_path`, `num_pr_region`, `rm_index_width`, `num_streamer`) from the `hw_builder` instance; any of them can still be passed explicitly to override the derived value, and the fully manual form without `hw_builder` remains supported.

Table 7 describes each parameter of `HwBuildHelper`.

Table 7: HwBuildHelper Parameters

Parameter	Description
<code>build_folder_path</code>	Path to the directory where Vivado project files and other temporary working files are written. The folder will be created if it does not exist.
<code>dfx_root_path</code>	Root path of the DFX4ML-ARCH source repository, used to locate IP cores, build scripts, and constraint files.
<code>board</code>	Target FPGA board identifier. Use a known board name (e.g., <code>kv260</code>) or <code>"custom"</code> to supply <code>board_build_tcl</code> and <code>constraint_xdc</code> manually.
<code>user_repo_path</code>	Path to the user-supplied IP repository containing partial-region kernels. Referenced only when <code>test_mode=0</code> .
<code>user_rm_build_tcl_path</code>	Path to the user-supplied Tcl script that builds the reconfigurable module (RM) IP. Referenced only when <code>test_mode=0</code> .

Parameter	Description
<code>req_gen_ip</code>	Flag (1/0) controlling whether Verilog IP generation is run before the build. The generated IPs are: ICAP interface, DFX Manager, DFX Streamer, and DFX Streamer Shut Plug. Must be set to 1 the first time or whenever <code>build_folder_path</code> has been deleted or modified.
<code>num_core</code>	Number of parallel synthesis/implementation jobs to run in Vivado.
<code>clk_frq</code>	Requested clock frequency in Hz (e.g., 99999001 $\approx$ 100 MHz).
<code>rm_index_width</code>	Bit-width of the RM index field, which sets the number of slots in DFX Mng Bank 1 and DFX Ctrl as $2^{\text{rm_index_width}}$ (e.g., <code>rm_index_width=2</code> allocates 4 slots). The actual number of loaded RMs is determined by <code>rm_schemetics</code> .
<code>dfx_streamers</code>	List of streamer configuration dicts, one per DFX Streamer instance. Index0 is always the DMA pass-through streamer. Each dict must contain: <code>load_width</code> (bus width in bytes for load path, power of 2), <code>store_width</code> (bus width in bytes for store path, power of 2), <code>actual_width</code> (effective data width in bits, $\leq$ bus width $\times$ 8), and <code>amount_row</code> (internal BRAM depth in rows).
<code>dfx_regions</code>	List of region configuration dicts, one per reconfigurable region. Each dict must contain: <code>load_streamers</code> (list of streamer indices that supply data to this region) and <code>store_streamers</code> (list of streamer indices that receive data from this region). Indices must be valid entries in <code>dfx_streamers</code> .
<code>rm_schemetics</code>	Two-dimensional list of RM schematic dicts indexed as <code>rm_schemetics[region_idx][rm_idx]</code> . Each dict must contain: <code>load_io_map</code> (list of (<code>streamer_index</code> , <code>kernel_port_index</code>) pairs for input connections) and <code>store_io_map</code> (list of (<code>streamer_index</code> , <code>kernel_port_index</code>) pairs for output connections). All regions must declare the same number of RMs.
<code>test_mode</code>	Flag (1/0) enabling test mode. When set, loopback logic is inserted for hardware verification without a user RM. Set to 0 to use the actual user kernel.
<code>vivado_path</code>	Absolute path to the Vivado executable (e.g., <code>/tools/Xilinx/Vivado/2023.2/bin/vivado</code>).
<code>export_folder_path</code>	Destination directory for packaged output files (<code>.bin</code> , <code>.hwh</code> , DFX Ctrl configuration, and PYNQ drivers). The folder will be created if it does not exist.

Parameter	Description
<code>board_build_tcl</code>	<i>(Optional; required when <code>board="custom"</code>)</i> Path to the board-specific Tcl build script. Ignored for known board identifiers.
<code>constraint_xdc</code>	<i>(Optional; required when <code>board="custom"</code>)</i> Path to the board-specific XDC constraint file. Ignored for known board identifiers.

4.2 ML Framework Integration

DFX4ML-ARCH provides the official solution for enabling Hardware Machine Learning frameworks to integrate their own sub-kernels into each Reconfigurable Module (RM).

To integrate the module, users must complete the following steps:

1. Provide the path to the user IP repository via `user_repo_path`. The folder at that path must contain the Vivado-exported IP folder, which in turn must include `src` and `xgui` sub-folders.
2. Provide the path to the TCL file that instructs the build script how to build the Reconfigurable Module via `user_build_tcl` (Listing 1).
3. Set `test_mode` to 0 (Listing 1) to enable the actual kernel instead of the loopback test logic.
4. Provide the IP name list in `ip_map_list` (Listing 1). The index corresponds to the Reconfigurable Module and the value is the IP name.

4.2.1 User Build Tcl

The user build TCL file must define the `create_dfx_region_bd` procedure with the following signature:

Listing 2: User Build TCL Procedure Signature

```

1 proc create_dfx_region_bd { \
2     parentCell \
3     block_name \
4     clk_frq \
5     interface_widths \
6     rm_config \
7     ip_name \
8     rm_id \
9 } {
10     # User implementation here
11 }
```

Table 8 describes each argument of the procedure.

Table 8: create_dfx_region_bd Arguments	
Argument	Description
parentCell	This field is deprecated.
block_name	Name of the hierarchical block representing the Reconfigurable Module in the block design.
clk_frq	Clock frequency in Hz passed from HwBuildHelper, used to configure IP timing constraints inside the region.
interface_widths	List of data-bus widths (bits) for each DFX Streamer, derived from dfx_streamers in HwBuildHelper. Each entry is the bus width in bits for the corresponding streamer index.
rm_config	RM schematic dict for this specific (region, RM) pair, corresponding to one element of rm_schematics in HwBuildHelper. Must contain two keys: load_io_map (list of {streamer_index kernel_port_index} pairs for input connections) and store_io_map (list of {streamer_index kernel_port_index} pairs for output connections).
ip_name	IP core name string for this kernel slot. An empty string indicates no additional IP is mapped.
rm_id	Zero-based integer identifier of the Reconfigurable Module within its region, used to uniquely name generated block design elements.

The argument `rm_config` is passed directly from the corresponding element of `rm_schematics` defined in `HwBuildHelper` (Listing 1).

Users must provide the RM implementation, which includes:

1. Create Block Design
2. Add ML IP module
3. Connect the I/O (clk, nreset (active low), AXI-Stream interface(s))
4. Validate Block Design
5. Close Block Design

Users may use [hw/bd_src/dfx_region/dfx_region.tcl](#) as a reference.

4.3 Board-Specific Script

In the current version, only the KV260 board can be fully synthesized and implemented. If the *board* parameter is set to *custom*, the hardware build will stop at the DFX block design with DFX Unified and partial connections; the PS and interconnect will be missing.

To define a new board, users must create or modify the following files:

1. **board_file**: users must create the file at `hw/build_script/<your board name>/board_build.tcl`
The purpose of this file is to provide a processor block, interconnect block, interrupt system, and other relevant IPs in the Vivado block design. Users can use `hw/build_script/kv260/board_build.tcl` or the GUI in board *custom* as a reference.
2. **constraint_file**: users must create one file per supported region count at `hw/build_script/<your board name>/constraint_<N>_region.xdc` where *<N>* is the number of reconfigurable regions; the build selects the file matching `num_dfx_region`. The purpose of these files is to define the boundaries of each partial reconfigurable region and its resources. Users can use `hw/build_script/kv260/constraint_1_region.xdc` and `hw/build_script/kv260/constraint_2_region.xdc` as references (the KV260 currently supports 1 and 2 regions).
3. **build_script**: users must modify the file at `hw/build_script/build.tcl` to invoke the block design creation procedure. Users may use the kv260 option as a reference.

The DFX4ML-ARCH authors would appreciate pull requests for board-specific files.

5 HLS4ML Integration

The DFX4ML-ARCH build flow described in Section 4 expects the user to hand-author each Reconfigurable Module (RM) as a Vivado-exported IP plus a Tcl block-design procedure, and to hand-write the three streamer configuration objects (`dfx_streamers`, `dfx_regions`, `rm_schematics`). For neural-network workloads this is error-prone: the kernel ports, the AXI-Stream bus widths, and the inter-region buffer geometry must all agree with the partitioned model. To remove that burden we provide a **hls4ml** [2] integration that turns a partitioned ML model directly into dfx4ml user kernels and the matching `HwBuildHelper` parameters.

The integration is delivered as two cooperating Python components plus two driver notebooks:

- `lib/hls4ml_con` — a hls4ml *backend plugin* that customises hls4ml’s code generation and IP packaging so each partition is emitted as a multi-port flat AXI-Stream kernel compatible with the DFX Streamer boundary (Section 5.1).

- `lib/hls4ml_build` — an *orchestrator* class, `Hls4ml_build`, that drives the whole Keras → hls4ml → dfx4ml flow (convert, csim, synthesis, streamer-glue) and hands off directly to `HwBuildHelper` / `SwBuildHelper` (Section 5.2).
- `quick_start_hls4ml_part_1.ipynb` and `quick_start_hls4ml_part_2.ipynb` — two end-to-end example notebooks (Section 5.3).

The hls4ml source itself is included as a Git submodule (`hls4ml/`) and is treated as read-only: all dfx-specific behaviour is injected from the plugin (via overridden templates and post-write patches), so the submodule never has to be forked.

5.1 The hls4ml_con Backend Plugin

hls4ml discovers external backends through the `HLS4ML_BACKEND_PLUGINS` environment variable; the user sets `os.environ['HLS4ML_BACKEND_PLUGINS'] = 'hls4ml_con'` *before* importing hls4ml. The plugin then registers a new backend named `VitisUnifiedDFx4ml` (lookup key `vitissunifieddfx4ml`) which extends hls4ml's stock `VitisUnified` backend. Selecting this backend during conversion is what makes hls4ml emit dfx4ml-compatible firmware. Table 9 lists the files that make up the plugin.

Table 9: Contents of `lib/hls4ml_con`

File	Role
<code>backend.py</code>	Registers the <code>VitisUnifiedDFx4ml</code> backend (subclass of <code>VitisUnified</code>). Establishes the conversion flow and default project configuration.
<code>writer.py</code>	<code>VitisUnifiedDFx4mlWriter</code> — emits each partition as a <i>multi-port flat AXI-Stream</i> kernel (one AXI-Stream port per streamer, each carrying <code>TKEEP</code> and <code>TLAST</code>), and drives C simulation (csim), co-simulation (cosim), and <code>ip_catalog</code> packaging. After hls4ml's own <code>write_hls()</code> runs it applies the post-write patches described below.
<code>streamer_glue.py</code>	<code>build_dispatcher_tcl()</code> stitches the per-(region,rm) Tcl fragments into the single user block-design dispatcher; <code>stream_geometry_from_hls()</code> reads each output port's width/depth back out of the generated HLS project.
<code>tcl_gen.py</code>	Renders <code>create_dfx_region_user_bd</code> from <code>dfx_region_user_bd.tcl.template</code> , emitting one IP VLNV fragment per (region,rm) pair.

File	Role
dfx_region_user_bd.tcl.template	The Tcl procedure body, with markers (<code>__VLNV_TABLE__</code> , <code>__KERNEL__</code> , <code>__CTRL_BUSIF__</code> , <code>__*_PIN__</code>) that <code>tcl_gen.py</code> substitutes.
templates/ vitis_unified/	dfx-modified kernel templates (<code>myproject_axi_stream.{cpp,h}</code> , <code>nnet_helpers_dfx.h</code>) that replace the stock hls4ml templates for this backend.

Multi-port kernel model. The standard hls4ml VitisUnifiedBackend AXI-Stream kernel exposes a single input and a single output stream. DFX4ML-ARCH instead routes each tensor that crosses a region boundary through its own DFX Streamer. The writer therefore generates one flat AXI-Stream port per streamer the kernel touches, in kernel-port order, each carrying TKEEP and TLAST (both mandatory at the RP boundary, see Section 4.2). A kernel port that a particular RM variant does not use is left idle and wired to a `Stream*_Dummy` so that every RM variant inside a region presents an identical interface to the static fabric — a requirement for partial reconfiguration.

Post-write patches. Because the hls4ml submodule is read-only, the writer mutates the generated firmware in place (in `<out>/firmware/nnet_utils/`) after `super().write_hls()` returns:

- `_patch_nnet_helpers_keeplast` — promotes the float AXI-Stream packet type to `hls::axis<float,0,0,0,24>` so that TKEEP and TLAST are present on the wire.
- `_patch_nnet_dense_resource_lutram` — for `reuse_factor > 1`, retargets the dense weight ROM from `ROM_nP_BRAM` to `ROM_1P_LUTRAM` across all three `dense_resource` specializations. The patch is guarded for existence and idempotency so it is safe to re-run on the FIFO re-convert path.

5.2 The Hls4ml_build Orchestrator

Hls4ml_build (in `lib/hls4ml_build`) wraps the entire flow as a single class so the user does not have to invoke hls4ml, csim, synthesis, and the streamer-glue computation by hand. It is organised as a small package: a core (`builder.py`: constructor, validation, tool paths) that composes per-stage mixins (`_convert.py`, `_csim.py`, `_synth.py`, `_glue.py`, `_diag.py`); `dfx_params.py` holds `compute_dfx_params`; and `topology.py` holds the typed topology helpers.

Typed topology. The cut is described with two dataclasses — `Partition` and `Stream` — which are the *single* topology representation consumed throughout the pipeline (there is no parallel dict schema). A `Partition` is one RM variant (kernel) placed at a (`region,rm`) coordinate; its `inputs` and `outputs` are ordered by kernel port, where each entry is either the sentinel DMA (the external DMA path, streamer index 0) or a `Stream`. A `Stream` declares only its bank-lifetime fields — `region` (which region owns the BRAM bank) and `alloc_phase / free_phase` (its lifetime in reconfiguration phases) — plus an optional `precision`. Crucially, everything else is *derived*, never hand-written: a stream’s `shape` is filled in by the producing `Partition` from the matching Keras model output (so a hand-typed shape can

never disagree with the model), and the geometry fields (`amt_banks_per_entry`, ...) are filled in later during bank allocation. The number of reconfiguration phases (`amt_phase`) and regions (`num_regions`) are likewise inferred from the topology.

Listing 3: Describing the cut with the typed topology helpers

```

1 from hls4ml_build import Hls4ml_build, Partition, Stream, DMA
2
3 parts = [
4     Partition('halfA', 'p_halfA', halfA, region=0, inputs=[DMA],
5               outputs=[
6                 Stream('bneck', region=0, alloc_phase=0, free_phase=1),
7                 Stream('skip2', region=0, alloc_phase=0, free_phase=1),
8                 Stream('skip1', region=1, alloc_phase=0, free_phase=1)]),
9     Partition('halfB', 'p_halfB', halfB, region=1,
10              inputs=['bneck', 'skip2', 'skip1'], outputs=[DMA]),
11 ]

```

Validation. On construction the orchestrator validates the topology: stream names must be unique, every non-DMA I/O name must resolve to a stream some partition produces, and there must be exactly one entry partition (a DMA input) and exactly one exit partition (a DMA output). It also warns (without failing) when partitions are not listed in producer-first order, since end-to-end csim assumes that order.

Stage methods. The orchestrator is constructed with the partition list, an output root, the conversion configuration (backend, precision, reuse factor, ...), and the Vitis / Vivado install directories. When those install dirs are given it sources each tool's `settings64.sh` onto `PATH` (`setup_env()`). The flow is then driven through the methods in Table 10.

Table 10: Hls4ml_build stage methods

Method	Purpose
<code>convert_all(fifo_opt=)</code>	Convert every partition's Keras model through hls4ml with the <code>VitisUnifiedDFx4ml</code> backend, producing one HLS <code>ModelGraph</code> per partition (the "partial model").
<code>csim_partition(name, inputs)</code>	Run C simulation on a single partition.
<code>csim_chain(x0=, n=)</code>	Run end-to-end C simulation across the whole chain of partitions, feeding each partition's output to the next, to validate the cut numerically before synthesis.

Method	Purpose
<code>diag_bisect(keras_model, probe_layers, x_input)</code>	Per-layer HLS csim bisect: converts a one-layer-deep probe model for each layer and reports where the fixed-point signal collapses or goes uniform — a diagnostic for precision problems.
<code>compute_streamer_glue()</code>	Compute the streamer assignment and emit the dispatcher Tcl; sets <code>self.dfx</code> (the <code>compute_dfx_params</code> result) and <code>self.user_rm_tcl</code> . Must be run before the hardware build.
<code>synth_all(fifo_opt=)</code>	Run C-synthesis and <code>ip_catalog</code> packaging for every partition, optionally with FIFO-depth optimization.

Streamer glue. `compute_dfx_params(partitions, ...)` (in `dfx_params.py`) is the heart of the automation. It walks each producer’s `Stream` objects, assigns streamers (index 0 is always the DMA), unions the load/store streamers per region, and builds the per-(region,rm) (`streamer_idx`, `kernel_port`) maps; a `kernel_port` of `-1` marks a port that the RM leaves idle, so a `Stream*_Dummy` is wired in its place. The result is exactly the trio of objects (`dfx_streamers`, `dfx_regions`, `rm_schemetics`) that `HwBuildHelper` consumes — it is the automated equivalent of the hand-written configuration in Listing 1.

Handoff to the build. Both build helpers accept the orchestrator instance directly and pull their configuration from it. `HwBuildHelper(hls4ml_build=hb, ...)` unpacks `dfx` into `dfx_streamers`/`dfx_regions`/`rm_schemetics` and uses `user_rm_tcl` as the user RM Tcl; `SwBuildHelper(hls4ml_build=hb, export_folder_path=...)` pulls the region and streamer counts. Explicitly-passed values still override the derived ones. The user must call `hb.compute_streamer_glue()` before the hardware build.

Listing 4: End-to-end flow with the orchestrator

```

1 from lib.hw_build import HwBuildHelper
2 from lib.sw_build import SwBuildHelper
3
4 hb = Hls4ml_build(parts, out_root='./hls4ml_dfx_out',
5                   vitis_path = '/tools/Xilinx/Vitis/2023.2',
6                   vivado_path = '/tools/Xilinx/Vivado/2023.2')
7 hb.convert_all()           # 1. get the partial model
8 hb.csim_chain()           # 2. (optional) end-to-end csim
9 hb.compute_streamer_glue() # 3. streamer glue (-> hb.dfx, hb.
   user_rm_tcl)
10 hb.synth_all(fifo_opt=True) # 4. C-synth + ip_catalog package
11
12 # hand off to the hardware/software build
13 hw = HwBuildHelper(build_folder_path='./build_prj', dfx_root_path='.',
   ,

```

```
14         export_folder_path='./export', test_mode=0,  
15         hls4ml_build=hb)  
16 hw.run_build(); hw.package_export_files()  
17 SwBuildHelper(hw_builder=hw).package_export_file()
```

5.3 Quick Start

Two example notebooks reproduce the flow above cell by cell and serve as the recommended entry points:

- [quick_start_hls4ml_part_1.ipynb](#) — configuration `config_2`: a single PR region holding two RM variants. The simplest end-to-end example.
- [quick_start_hls4ml_part_2.ipynb](#) — configuration `config_4`: two PR regions with four RM variants total, demonstrating a multi-region pipeline with inter-region streams.

References

- [1] Dfx controller v1.0 product guide (pg374). <https://docs.amd.com/v/u/en-US/pg374-dfx-controller>, 2026.
- [2] FastML Team. `fastmachinelearning/hls4ml`, 2026.